

MixBytes()

[SECURITY AUDIT REPORT]

Gearbox core-v3 Account Factory Diff

PREPARED FOR: GEARBOX

REPORT DATE

August 07, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	10
2.1 Critical	10
2.2 High	11
H-1 Funds can become locked after market expiration because Credit Account closure always reverts	11
2.3 Medium	12
2.4 Low	12
3. About MixBytes	13

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Gearbox users interact with the protocol through Credit Accounts. These smart-contract wallets hold the user's collateral and borrowed funds and access approved DeFi protocols through adapters. Each credit manager has an account factory; the Credit Facade and Credit Manager handle opening, multicalls, liquidation and closing.

This audit covers one change to the default account factory. The previous factory reused closed accounts after a three-day delay. It kept them in per-manager queues and allowed the DAO to call `rescue` while they were queued. The new factory deploys a fresh clone for every account, always reverts when an account is returned, and no longer uses `Ownable` or `rescue`.

The change is intended to support RWA integrations that attach external state to an account address. Examples include compliance roles, redemption requests and settlements delivered without a callback. Reusing an address could expose that state to the next borrower. However, closing a Credit Account currently requires returning it to the factory, so the unconditional revert also disables closing.

We traced the affected call paths in `core-v3` and its dependent repositories, reproduced the new behaviour in the project's Foundry environment, and tested account lifecycles and exit paths with property-based and stateful fuzzing. We also checked how the affected functions are used on-chain.

Scope Limitations

The audit scope is limited to the reviewed change in `DefaultAccountFactoryV3`. All other code is out of scope. Other `core-v3` contracts and the `permissionless`, `periphery-v3`, and `bots-v3` repositories were inspected only as context for the affected call paths. The RWA integrations are covered by a separate audit.

Review Coverage

The review covered the following questions specific to this change:

- Can users remove funds through withdrawals, closing, full or partial liquidation, or governance actions while a market is live, paused or expired?
- Do new clones reference the correct credit manager and factory, and is all required initialization still performed?
- Can the same account address be issued more than once?
- Can someone block credit-suite deployment by deploying to the factory's deterministic address first?
- Can an existing market adopt the new factory without affecting open accounts?

Additional Notes

- **Removal of the DAO rescue path.** We found no separate asset-loss scenario caused by this removal. Under the new design, an account remains with its owner. The owner can recover the assets, except in the case described in the finding below.
- **Closing still exists but no longer works.** The function can be called, but it reverts inside the factory. This wastes gas and returns an unclear error. The recommended fix below restores closing; if the current behaviour is retained, the Credit Facade should reject the call directly with a dedicated error.
- **Migration requires a new deployment.** `accountFactory` is immutable in the Credit Manager and has no setter. Suite deployment always creates a new factory and cannot attach an existing one. Existing markets therefore keep their original factory and are not affected by this change. Any integration that requires unique account addresses must use a new credit suite.
- **Tests do not cover accounts that hold funds after market expiration.** Existing tests check that individual functions revert after expiration and that the expiration date is configured correctly. They do not test whether a funded account can still withdraw after expiration.
- **Restoring closing must not reintroduce address reuse.** One option is to allow closing only when every enabled collateral token has a zero balance. Phantom tokens include pending positions in `balanceOf`, so this check would prevent finalization while a delayed position remains outstanding without blocking ordinary closure.

The audit identified one High-severity issue, described below. We found no other case in which funds become inaccessible. We also confirmed the reason for removing account reuse: under the previous design, an account could be closed with a position still attached to its address and later assigned to another borrower.

During the re-audit, commit `d47bf89eeb87ffa8699ed73f75892dc68f6fae14` changed `returnCreditAccount` to a no-op and restored the owner-controlled `rescue` function while preserving the fresh-clone model. This restores Credit Account closure, including withdrawals after market expiration, without allowing closed addresses to be issued again. The High-severity finding is considered fixed, and no new security issues were identified in the updated code.

1.3 Project Overview

Summary

CLIENT	Gearbox Protocol
CATEGORY	Lending
PROJECT	core-v3
TYPE	Solidity
PLATFORM	EVM
TIMELINE	03.08.2026 - 07.08.2026

Scope of Audit

```
contracts/core/DefaultAccountFactoryV3.sol
```

Versions Log

DATE	COMMIT HASH	NOTE
03.08.2026	510fc6541c3767ce825929b4c311826fe81d6fa5	Diff Base
03.08.2026	b2d6a42874f73fa3d3c17402bfa230ea628f663b	Initial Commit
04.08.2026	d47bf89eeb87ffa8699ed73f75892dc68f6fae14	Re-audit Commit
06.08.2026	f162ab98fd97d67484bf9ad209b4dc41116042bb	Minor Comment Fix Commit

Mainnet Deployments

Deployment verification will be conducted via <https://permissionless.gearbox.foundation/bytecode/>

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

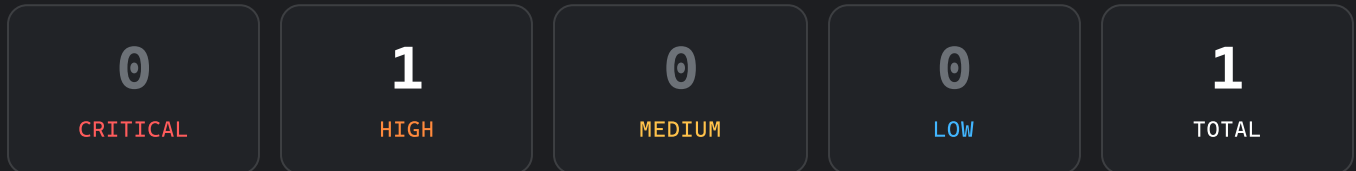
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
H-1	Funds can become locked after market expiration because Credit Account closure always reverts	● HIGH	● FIXED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

H-1 Funds can become locked after market expiration because Credit Account closure always reverts

● HIGH

● FIXED AS OF d47bf89e

DESCRIPTION

Funds may remain in a Credit Account when its market expires. After expiration, `whenNotExpired` blocks regular owner and bot multicalls. The only remaining user exit paths are `closeCreditAccount`, `liquidateCreditAccount`, and `partiallyLiquidateCreditAccount`, but both liquidation functions revert at zero debt in `_revertIfNotLiquidatable` (`CreditFacadeV3.sol:975-983`). A user with a zero-debt account must therefore call `closeCreditAccount` and include the required withdrawal operations to recover those funds (`CreditFacadeV3.sol:251-275`, `448-474`).

The new factory makes this exit unusable. `CreditManagerV3` calls `accountFactory.returnCreditAccount` while closing an account (`CreditManagerV3.sol:250-288`), but the updated factory always reverts in that function (`DefaultAccountFactoryV3.sol:53-56`), blocking the withdrawal operations. A user with funds in an expired market cannot withdraw them without an administrator changing the market configuration.

The configurator can make withdrawals possible again, but only by changing the whole market. Extending `expirationDate` reopens the market for at least two weeks. Replacing the Credit Facade works only if the new facade is not expirable, because a like-for-like replacement inherits the old expiration date. An existing market cannot restore closing itself: the revert occurs when its Credit Manager calls the factory, and the manager's `accountFactory` reference is immutable.

This exit is used in practice. On-chain history contains closes on several facades after their expiration timestamps, and each close included a `withdrawCollateral` call.

Likelihood is High. Any expirable market reaches this state with time, without an attacker or unusual user action, and the issue affects each funded, zero-debt account at expiration. On-chain history confirms that users rely on post-expiration closing. Impact is Medium because the funds are locked, not lost, and an administrator can recover access by changing market-wide configuration. Medium impact and High likelihood result in High severity.

RECOMMENDATION

We recommend allowing the Credit Manager to finalize an account without adding it to a reuse queue, or add a non-reverting finalization function to the factory. Closing must work without allowing the address to be issued again.

CLIENT'S COMMENTARY

Fixed in d47bf89eeb87ffa8699ed73f75892dc68f6fae14. `returnCreditAccount` is now a no-op and `rescue` is restored to save potential stranded user funds.

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

NO LOW SEVERITY FINDINGS

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

